

一. 问题分析

1. 分析并确定要处理的对象是什么

本实验中输入的是入轨道车厢的一个排列，要输出需要最少的缓冲轨道数量。

所以处理的对象是输入的数字序列

2. 分析并确定要实现的功能是什么

给定一组车厢编号的排列，求出在“重排”所需的最少缓冲轨道数。这里的“缓冲轨道”可以理解为临时存放车厢的队列，要求在车厢调度完成后，输出的车厢序列满足从左向右递减的顺序

所以实现的功能是求得缓冲轨道数

3. 分析并确定处理后的结果如何显示

程序最终输出一个整数，表示该车厢排列所需的最少缓冲轨道数量。

4. 用题目中样例，详细给出样例求解过程。

题目的样例输入为：5 8 1 7 4 2 9 6 3

4.1 初始状态 **trackTops**（缓冲轨道数组）为空

读入车厢 5；**trackTops** 中没有轨道（为空），因此无法找到“轨道末尾大于 5”的情况。

新建一条轨道，放入 5。

更新 **trackTops** 为：[5]

4.2：读入车厢 8，遍历轨道，轨道 0 的末尾为 5，判断 $5 > 8$ ？不符合。

没有找到合适的轨道，将 8 放入新轨道。

更新 **trackTops** 为：[5][8]

4.3 读入车厢 1，遍历轨道，轨道 0 的元素 $5 > 1$ ，符合，所以将车厢 1 放入轨道 0，更新轨道末尾为 1。

标记当前车厢已放置，跳出循环。

更新 **trackTops** 为：[1][8]

4.4 读入车厢 7，遍历轨道，轨道 0 的末尾为 1，判断 $1 > 7$ ？不符合。

轨道 1 的末尾为 8，判断 $8 > 7$ ？符合！

将车厢 7 放入轨道 1，更新轨道末尾为 7。

标记当前车厢已放置，跳出循环

更新 **trackTops** 为：[1][7]

4.5 读入车厢 4，遍历轨道，轨道 0 的末尾为 1，判断 $1 > 4$ ？不符合。

轨道 1 的末尾为 7，判断 $7 > 4$ ？符合！

将车厢 4 放入轨道 1，更新轨道末尾为 4。

标记当前车厢已放置，跳出循环

更新 **trackTops** 为：[1][4]

4.6 读入车厢 2，遍历轨道，轨道 0 的末尾为 1，判断 $1 > 2$ ？不符合。

轨道 1 的末尾为 4，判断 $4 > 2$ ？符合！

将车厢 2 放入轨道 1，更新轨道末尾为 2。

标记当前车厢已放置，跳出循环

更新 **trackTops** 为：[1][2]

4.7 读入车厢 9，遍历轨道，轨道 0 的末尾为 1，判断 $1 > 9$ ？不符合。

轨道 1 的末尾为 2，判断 $2 > 9$ ？不符合。

没有找到合适的轨道，将 9 放入新轨道。

更新 `trackTops` 为：[1][2][9]

4.8 读入车厢 6，遍历轨道，轨道 0 的末尾为 1，判断 $1 > 6$ ？不符合。

轨道 1 的末尾为 2，判断 $2 > 6$ ？不符合。

轨道 2 的末尾为 9，判断 $9 > 6$ ？符合！

将车厢 6 放入轨道 2，更新轨道末尾为 6。

标记当前车厢已放置，跳出循环

更新 `trackTops` 为：[1][2][6]

4.9 读入车厢 3，遍历轨道，轨道 0 的末尾为 1，判断 $1 > 3$ ？不符合。

轨道 1 的末尾为 2，判断 $2 > 3$ ？不符合。

轨道 2 的末尾为 6，判断 $6 > 3$ ？符合！

将车厢 3 放入轨道 2，更新轨道末尾为 3。

标记当前车厢已放置，跳出循环

更新 `trackTops` 为：[1][2][3]

至此，完成了所有入轨车厢的读入，最终得到三个“队列”，所以至少需要 3 条缓冲轨道。

二. 数据结构与算法设计

1. 抽象数据类型设计

ADT 名称：BufferTrack

数据对象：一组整数序列 `vector<int> sequence` 用来存储入轨车厢编号

一个数组 `vector<int> trackTops` 用来存储每个缓冲轨道当前尾部车厢编号

基本操作：BufferTrack() 构造函数

`computeTrackCount(sequence)` 提供用于输入入轨序列计算所需最小缓冲轨道数的核心接口。

2. 物理数据对象设计：

“物理存储结构”是一个**动态数组**，将动态数组“`trackTops`”当作抽象的线性表使用，它提供了在末尾增添元素、访问和修改指定下标元素等操作。

在实际实现中：

头文件 `buffertrack.h` 中声明了 BufferTrack 类，包含 `vector<int> trackTops` 作为私有成员，用来存储各条轨道“末尾”的车厢编号

源文件 `bufferstack.cpp` 中实现了 BufferTrack 的构造函数和 `computeTrackCount` 方法。

主函数文件 `main.cpp` 负责读取输入、调用 `computeTrackCount`、并输出结果。

3. 算法思想的设计。

我分别实现了两种算法，一种是题目给出的直接模拟队列的算法，一种是优化后的简便求法，下面将两种算法分别阐释。最终选取优化后的算法作为报告算法。

3.1 题目给出的直接模拟队列算法的分析：

题干给出的操作过程可通过以下算法实现，该算法分为以下几个部分

第一部分：需要先遍历输入车厢序列，每个车厢都会进入处理流程

第二部分：初始化 nowout（当前允许出轨的编号）为 1，遍历所有轨道，检查队首是否等于 nowout 以判断是否有可以出轨的列车，出轨一辆车，nowout 自增一次。

第三部分：每当在某个队列发现队首车厢能直接出轨时，会执行出队操作并将索引重置，从而使得外层对所有队列的检查必须重新开始。这种重置操作可能导致内层循环重复执行多次，进一步增加了总的计算量。

第四部分：当车厢不能直接出轨时，算法需要遍历所有缓冲轨队列，查找出队尾元素小于当前读取的入轨车厢且最大的队列。若没有这样的队列，还需要新开一个队列。

总的来看，第四部分是在第一、二部分执行失败的情况下执行的，而第三部分为第二部分的后继处理。

由此来看，这种算法直接模拟了列车进出队列的过程，需要多次操作与判断，不同部分的算法有时并行处理，实现或理解都较为复杂

3.2 优化的算法思想介绍

3.2.1 算法思想描述

题干给出的重排方法基础上优化得到一种更为简洁的方法，该方法只关心每个轨道的最后一个元素，通过读入序列持续更新队列末尾元素或增加轨道的方式求解问题，最终得到队列数就是所需缓冲轨数目。

该算法运用贪心算法，我们使用 trackTops 数组维护所有当前存在轨道，trackTops[i] 表示某一个缓冲轨道的末尾车厢编号，依次读取车厢编号（car）。在已知轨道中查找到某个轨道满足当前末尾编号大于该车厢编号，则将该轨道末尾编号更新为 car

如果找不到合适的轨道，就新建一条轨道，trackTops 的元素个数加一。

最后 trackTops 的元素个数就是需要缓冲轨道的数量。

3.2.2 算法思想原理

贪心策略：

每次处理一个车厢时，尽可能将其放入已有的轨道中，并更新该轨道的末尾为更小的值。这样可以为后续车厢留出更多“插入”空间，最终使得缓冲轨道数最小。

耐心排序：

该排序算法的基本思路是将输入序列分成若干个有序的子序列，然后通过合并这些子序列来得到最终的有序序列。

这里的输入序列就是样例给定的车厢编号；若干个有序子序列就是每条缓冲轨道上的序列，最终的有序序列就是重排后的序列。

算法通过不断地“更新”轨道末尾，实际上将输入序列分成若干个递增子序列，序列数就是所需轨道数

算法数学原理：

这个算法由 Dilworth 定理支撑其正确性。

Dilworth 定理：“对于任意有限偏序集，其最大反链中元素的数目必等于最小链划分中链的数目”。

组合数学的一个结论为：

给定一个序列，将其划分为最少的递增子序列的数量，恰好等于该序列的最长递减子序列的长度（这实际上是 Dilworth 定理的一个特例），也就是说：**最少递增子序列的数量与最长递减子序列的长度是等价的**

因为在一个递增子序列中，无法包含递减的元素，而递减子序列的元素会在不同的递增子序列中分布开来。最长递减子序列的长度决定了最少的递增子序列数，因为它揭示了序列中最“强”的递减趋势，即需要分开来存放的最大元素数量。

换句话说，**最终新建的轨道数就是对输入序列进行最优划分后所得到的递增子序列数。**

核心代码如下：

```
for (int car : sequence) {
    bool placed = false;
    // 尝试将当前车厢放入已有的缓冲轨道
    for (size_t i=0;i<trackTops.size();++i) {
        if (trackTops[i]>car) {
            //找到一个轨道，其尾部车厢编号大于当前车厢且是大于当前
            //车厢编号的最大编号，将当前车厢放入该轨道
            trackTops[i]=car;
            placed=true;
            break;
        }
    }
    if (!placed) {
        // 没有合适的轨道容纳当前车厢，新建一条缓冲轨道
        trackTops.push_back(car);
    }
}
```

4. 关键步骤的算法步骤。

4.1. 创建一个空的动态数组（`vector<int> trackTops`），用于存储当前每条缓冲轨道的“末尾”车厢编号。

4.2. 依次处理序列中的每个车厢，执行以下操作

4.2.1 遍历已建立的所有轨道，查找是否存在一个轨道，其末尾的编号大于当前车厢。

4.2.2 如果找到这样的轨道，说明当前车厢可以放在这个轨道上，更新该轨道的末尾编号为 `car`。这样做能使得轨道末尾变得更小，为后续更大的车厢提供更大的插入空间。

4.2.3 若未能放入任何轨道，新建一条轨道，并将当前车厢放入这条轨道中，作为数组末尾。

4.3 输出结果：

当所有车厢处理完毕后，`trackTops` 数组的大小即为整个序列划分成的递增子序列数，也就是所需的最少缓冲轨道数。

三. 算法性能分析

时间复杂度：

外层循环：依次遍历输入序列的每个车厢，时间复杂度为 $O(n)$

内层循环：在最坏情况下，trackTops 中的轨道数可能接近 n （例如，当输入序列呈严格非递增时，每个新元素都不能放入已有轨道，每次都需要新建一条轨道），因此内层循环的每次迭代可能需要扫描 $O(n)$ 个轨道。

整体来看，算法最坏时间复杂度为 $O(n) \times O(n) = O(n^2)$

空间复杂度：

输入序列的存储：

输入的车厢排列序列需要存储在一个数组或容器中（`vector<int>`）。这个数组的空间复杂度是 $O(n)$ ，其中 n 是输入车厢的数量。

轨道末尾数组：

主要空间消耗来自 trackTops，在极端情况下（序列严格递增），trackTops 大小可达 n 。因此空间复杂度为 $O(n)$ 。

总体空间复杂度是由输入数据和 trackTops 决定的，因此空间复杂度为 $O(n)$ 。

四．算法设计的反思：

优点：实现方法简单，思路清晰，代码量相较直接模拟队列大量减小。

在数学上能保证求得最小轨道数（有数学定理支撑）

如果对 trackTops 维护的轨道末尾采用二分查找整体时间复杂度则可以优化为 $O(n \log n)$ 。

缺点：采用线性搜索队列末尾元素，总体最坏时间复杂度仍为 $O(n^2)$ ，与题干所给直接模拟算法一致。

相较题干队列相比队列模拟算法，贪心方法并没有完全模拟真实的 FIFO 出轨过程，可能展示列车重排具体的流程不是很直观，本算法以求得轨道数结果为导向，但结果是等价且正确的。